

AI 導盲眼鏡 (Guide Glasses)

專題詳細計畫書

現況盤點、系統架構與後續執行計畫

項目	內容
版本	v2.0 (Cloudflare 版完成實測)
日期	2026 年 10 月 1 日
專案位置	glasses-project (分支 restructure/three-versions)
裝置	Rokid Glasses (Android 12) + Android 手機 (小米 · Android 13)
後端	FastAPI (家中電腦) + Cloudflare 通道

一、計畫摘要

本專題開發一套給視障者使用的 AI 導盲眼鏡系統。使用者戴上智慧眼鏡後，**不需要看螢幕、也不需要觸碰任何按鈕**，只要用說的，就能辨識前方障礙物、認出已登錄的人、朗讀文字，並完成「查詢公車 → 走到上車站 → 核對公車號碼」的完整出行流程。

系統由三個部分組成：眼鏡負責影像辨識與語音互動；手機提供 GPS 定位並當作網路閘道；家中電腦上的後端整合 LLM、Google Routes、TDX 與 Google Cloud Vision 等外部服務。安全相關的功能（障礙物、人臉、文字）全部在眼鏡上離線執行，只有需要即時資料的查詢才上網。

目前進度

指標	現況
主要功能	13 項中 10 項已在眼鏡上實測可用；確認公車待真實公車驗證、換網址的手機端待實測、翻譯待修
脫離線材	已完成：眼鏡經 Wi-Fi + Cloudflare 通道運作，不需要 USB
自動化測試	Android 單元測試 423 個、後端測試 15 個，全部通過
版本控制	所有完成的修改都已分組 commit（未 push）；金鑰與個資不進版控
三種版本	端側版（凍結）、Cloudflare 版（主力）、AWS 版（進行中，約 20%）

二、背景與目標

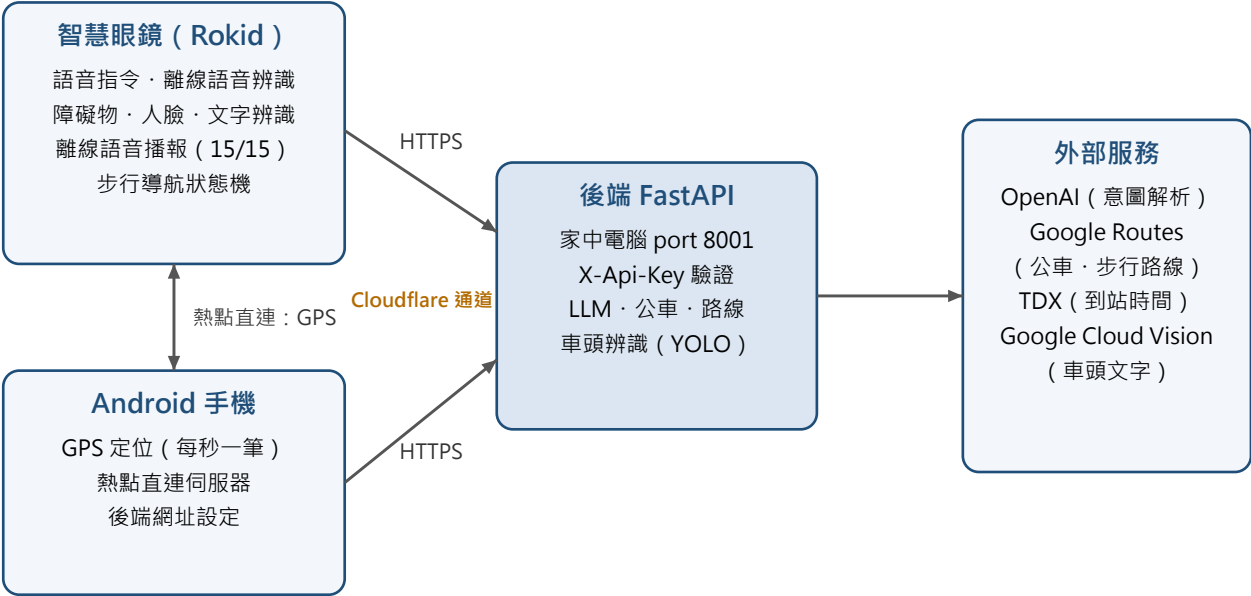
問題

視障者出門最大的困難，是無法即時掌握周遭狀況：前方有沒有障礙物、迎面走來的是誰、招牌寫什麼，以及最困難的「搭公車」——要知道該搭哪一路、站牌在哪、來的是不是自己要搭的車。現有輔具多半只解決單一問題，而且常要求使用者操作手機畫面。

目標

- **全語音操作**：所有功能用說的觸發，畫面上沒有按鈕；系統以語音與提示音回應。
- **安全功能離線**：障礙物、人臉、文字辨識在眼鏡上執行，沒有網路也能用。
- **完整公車出行**：從查路線、步行導航到上車站，到核對車頭號碼，一條龍完成。
- **可靠與安全**：網路中斷時有備援路徑；後端公開時有金鑰保護；人臉資料只存在裝置上並加密。

三、系統架構



三端分工

部分	負責什麼	主要技術
智慧眼鏡	語音指令偵測、語音辨識、影像辨識、語音播報、導航狀態機、播報仲裁	Kotlin、CameraX、sherpa-onnx、ONNX Runtime、ML Kit、Hilt
Android 手機	每秒取得 GPS；透過熱點直接提供給眼鏡；同時送給後端當備援	Fused Location、前景服務、區網 HTTP 伺服器
後端 (電腦)	LLM 意圖解析、公車方案、到站時間、步行路線、車頭辨識、定位轉傳	FastAPI、OpenAI、Google Routes、TDX、YOLO、Google Vision
對外連線	讓走 4G 的眼鏡與手機連到家中電腦	Cloudflare 通道 (HTTPS)，後端只聽本機

眼鏡上的 AI 模型

用途	模型 / 引擎	執行位置
語音指令 (18 句)	sherpa-onnx 關鍵詞偵測	眼鏡 (離線)
自由語句辨識	串流 zipformer CTC (中文・26 MB)	眼鏡 (離線)
語音播報	Matcha-TTS 中文、Piper 英文 (22050 Hz)	眼鏡 (離線)
障礙物偵測	YOLOv8n-seg (8 類)	眼鏡 (離線)
人臉辨識	ML Kit 偵測 + InsightFace w600k_mbf 特徵	眼鏡 (離線)
文字辨識 / 翻譯	ML Kit 中文 OCR / ML Kit 翻譯	眼鏡 (離線・翻譯需先下載語言包)
意圖解析	OpenAI gpt-4o-mini (function calling)	後端 → 雲端
車頭號碼	YOLO (LED 看板) + Google Cloud Vision	後端 → 雲端

定位資料流

眼鏡本身沒有 GPS。手機每秒取得一筆定位，眼鏡優先透過熱點直接向手機詢問（延遲毫秒級，4G 斷線也能用），連不到時才經由後端轉傳。每筆座標都帶有「多舊」（age_ms），以各裝置自己的開機時間計算，不依賴裝置的時鐘——實測眼鏡的時鐘快了 4 小時 15 分。超過 15 秒的座標一律丟棄，避免用舊位置導航。

四、三種版本

版本	眼鏡上的名稱	後端	狀態
端側版	導盲眼鏡	不連網	凍結（只保留，不再開發；不可與 CF 版同時開）
Cloudflare 版	導盲眼鏡 CF	家中電腦 + Cloudflare 通道	主力版本，已實測
AWS 版	導盲眼鏡 AWS	AWS Lambda（Learner Lab，US\$50）	進行中，約 20%

三個版本使用不同的 applicationId 與同一把簽章，可以同時裝在眼鏡上；被叫出的版本會透過受簽章保護的廣播，請其他版本讓出相機與麥克風。端側版是舊版本，沒有這個機制，所以拍攝與實測時要確認它沒有在執行。

五、功能清單與進度

下表的「狀態」以實機測試為準：「實測可用」代表已在眼鏡上完整跑過。

功能	怎麼操作（直接說）	需要網路	狀態
障礙物偵測	前面有什麼 / 持續偵測前方	否	實測可用
人臉辨識	這是誰 / 這個人是誰 / 前面有沒有人	否	實測可用
文字朗讀	上面寫什麼 / 唸給我聽 / 這是哪裡 / 下一段	否	實測可用
自由發問（LLM）	我要說話 →（欸）→ 我要搭公車去中正紀念堂	是	實測可用
查公車路線	查公車路線	是	實測可用
步行導航到上車站	查完公車後自動開始	是	實測可用
確認公車	確認公車	是	流程可用，待真實公車
停止播報	停止播報 / 不要說了	否	實測可用
出門前檢查	出門前檢查	否	實測可用
同步人臉	同步人臉（在家，接 USB）	區網	2 人 4 張
觸控板開始聆聽	點一下觸控板（不用滑動）	看指令	實測可用
翻譯	翻成英文	否（需語言包）	待修
換後端網址免重建	手機 App 貼新網址；眼鏡自動同步	是	眼鏡端完成，手機待實測

整合面的完成項目

- 免按鈕操作：拿掉畫面上的「說話」按鈕；新增語音指令「我要說話」；開 App 即要求權限。
- 脫離線材：眼鏡經 Wi-Fi 與 Cloudflare 通道運作，實測時移除所有 USB 轉接，各功能仍正常。
- 播報音量最大：改走導航語音音量，App 啟動時自動開到 15/15（原本只有 8/15）。
- 手機直連定位：眼鏡優先向手機直接拿 GPS；連續 3 次失敗才暫停 10 秒，改經由後端。
- 後端安全：除健康檢查外一律需要 X-API-Key；沒設定金鑰時拒絕所有請求（fail-closed）。
- 三版共存：共用簽章、不同套件名稱、感測器交接。

六、實測中解決的關鍵問題

許多問題在電腦上無法重現，只有上機才會出現，而且常常是「安靜地失敗」——沒有錯誤訊息，只是沒有反應。以下是影響最大的幾項：

問題	原因	解法
眼鏡沒有語音辨識、TTS 與 GPS	裝置宣告有，實際沒有	辨識與合成改用 APK 內建的離線引擎；定位改由手機提供
座標每一筆都被判定過期	眼鏡時鐘快 4 小時 15 分；室內定位本身就舊 7.5–12.8 秒	改用各自開機時間累加的 age_ms；門檻調為 15 秒
「我要說話」後前 6 秒收不到	語音辨識模型第一次使用才載入（6.4 秒）	啟動後背景預載；提示音改在真正開始聆聽時才響
說「中正紀念堂」查不到路線	辨識結果是簡體加同音錯字「中证纪念堂」	辨識結果轉繁體；LLM 修正地名為正式名稱
確認公車每次都失敗	金鑰檔是相對路徑，從其他資料夾啟動就找不到；缺 OCR 套件	路徑一律以 backend/ 為準；補齊並固定套件版本
確認公車唸出除錯文字	直接播報後端的 message（含 StopID、Direction）	由比對結果自行組句，只說「號碼相符」
播報太小聲	無障礙音量在眼鏡上固定 8/15 且無法調整	改走導航語音（媒體音量），啟動時開到最大
兩個版本同時回應	舊的端側版沒有感測器交接機制	拍攝前強制停止端側版；列入檢查表
通道重開就要重新建置	Quick Tunnel 網址每次都變	執行期可換網址；手機設定、眼鏡自動同步或 adb 設定
設定的網址重開 App 後遺失	設定以背景方式寫入，App 立即被關閉就沒存到	改為立即寫入，重新實測通過

七、測試與驗證

層級	內容	結果
Android 單元測試	語音指令一致性、公車播報、定位新舊判斷、手機直連、網址改寫等	423 個全過
後端測試	金鑰驗證、路徑解析、確認公車錯誤處理、辨識畫面存檔	15 個全過
實機自動化測試	以開發用廣播逐項觸發功能，確認眼鏡實際播報內容	全部通過
脫離線材測試	移除所有 USB 轉接，只走 Wi-Fi + Cloudflare 通道	全部通過
使用者實測	語音指令、我要說話、人臉、查公車、確認公車、停止、觸控板	順利（翻譯除外）

每次 commit 前都會自動檢查：沒有 .env、local.properties、金鑰檔被加入；實際金鑰值與常見金鑰格式不出現在內容中。

八、後續執行計畫

階段一：拍攝前收尾（約 1-2 天）

#	工作	內容	預估	需要
1	手機端換網址實測	安裝新版手機 App，測網址欄位與眼鏡自動同步，完成後 commit	0.5 小時	手機接 USB
2	修正翻譯	確認英文語言包的下載流程與失敗原因	1-2 小時	網路
3	拍攝目的地	把「查公車路線」的固定目的地設為拍攝地點，作為語音備案	0.5 小時	目的地
4	驗證 YOLO 框	以真實公車車頭照片確認 LED 看板偵測與存圖	1 小時	3-5 張照片
5	戶外實走	小米手機開熱點，實走步行導航並核對公車	2 小時	戶外
6	觸控板代碼	確認 Rokid 觸控板送出的按鍵代碼	0.5 小時	眼鏡

階段二：影片拍攝與剪輯

依《三分鐘專題影片企畫書》進行。旁白與字幕須依實際系統修正：後端是 FastAPI (Flask 已整併) ；GPS 由手機直接傳給眼鏡；步行導航由眼鏡語音引導（不開 Google Maps）；系統自動選擇最快方案；公車辨識只說「號碼相符」。辨識框畫面使用後端存下的真實結果，不以後製憑空繪製。

階段三：拍攝後強化（約 2–3 週）

優先	工作	說明
高	AWS 版完成	Mangum 包裝、DynamoDB 存座標、部署腳本；實測 Learner Lab 關閉時 Function URL 是否仍可用
高	固定網址	購買網域並改用 Cloudflare 具名通道，網址不再變動（約 US\$10 / 年）
中	準確度量測	障礙物偵測率；建立 30 句以上的 LLM 意圖測試集
中	手機直連不限熱點	眼鏡在任何 Wi-Fi 下都能找到手機（例如由後端轉告手機的區網位址）
中	開機自動啟動	研究 Device Owner / 廠商佈建，眼鏡開機直接進入導盲模式
低	持續整合	push 後自動建置與執行全部測試

九、風險與對策

風險	影響	對策
電腦或通道中斷	公車、LLM、確認公車無法使用	拍攝當天全程不關；障礙物、人臉、文字仍可離線運作
通道網址改變	App 連不到後端	已可在手機上換網址、眼鏡自動同步，不必重新建置
熱點不是手機開的	眼鏡找不到手機，定位改經後端（較慢）	拍攝時由小米手機開熱點
端側版同時執行	兩個聲音重疊、搶相機	開 CF 版前確認端側版已停止
戶外噪音	語音指令偵測不到	觸控板點一下作為備用；指令設計為四音節以上
公車班次稀疏	等不到目標公車	選班次密集的路線與時段；先拍其他段落
外部 API 額度	費用或配額用盡	後端金鑰驗證；只在使用者主動查詢時呼叫
個資	人臉照片、位置洩洩	人臉特徵只存眼鏡且加密；照片與位置資料不進版控；位置僅存最新一筆

十、啟動與操作手冊

拍攝 / 實測前的啟動順序

步驟	動作
1	電腦啟動後端：start-backend.ps1 -Port 8001
2	電腦啟動通道：cloudflared tunnel --url http://localhost:8001（網址若改變，在手機 App 貼上新網址）
3	小米手機開啟熱點，眼鏡連上；打開手機的「導盲定位 CF」
4	確認眼鏡上「導盲眼鏡」（端側版）沒有在執行
5	打開眼鏡的「導盲眼鏡 CF」，等約 20 秒讓語音模型載入
6	說「出門前檢查」，確認可用的功能

語音指令一覽 (直接說，不需喚醒詞)

類別	說法
停止	停止播報、不要說了
環境	前面有什麼、持續偵測前方
人臉	這是誰、這個人是誰、前面有沒有人、同步人臉
文字	上面寫什麼、唸給我聽、這是哪裡、下一段、翻成英文
出行	查公車路線、確認公車
系統	出門前檢查、測試相機
自由發問	我要說話 (聽到嗶聲後說出任何問題或目的地)

十一、專案結構

資料夾	內容
apps/app	眼鏡 App (CF / AWS 兩種建置)：畫面、前景服務、功能組裝、音量、網址改寫
apps/companion	手機 App：GPS 回報、熱點直連伺服器、後端網址設定
apps/core	功能邏輯 (查公車、確認公車、導航、語音指令、播報仲裁)、人臉資料庫
apps/ai	語音辨識、語音合成、LLM、障礙物、人臉、文字、翻譯、定位與公車
apps/glasses、feature	相機、動作感測；助理的指令分派與提示音
backend	FastAPI 後端、測試、Cloudflare 部署說明、AWS 版 (進行中)
edge	端側版 (凍結)
docs、shared、backup	文件、共用說明、整理前的舊內容